로봇 시뮬레이션과 제어 기초

로봇공학과 현실 격차 · 로봇 환경 탐색 실습 · PPO 연속제어

Machine Learning 2 · 13주차

한국과학영재학교 (KSA)

Week 13

이번 주차 개요

이 주차를 마치면

1. 로봇공학 3대 기둥, 시뮬레이션 한 스텝 = 시간 적분
2. **sim-to-real gap**의 원인, 도메인 무작위화 vs 시스템 ID
3. 연속 제어 평가 프로토콜, PD 제어 베이스라인과 비교
4. PPO를 **연속(가우시안) 정책**으로 확장해 Reacher 학습

- **13.1** 2관절 팔 FK/IK 손계산 · 진자 동역학 · PD 제어 · 현실 격차
- **13.2** PD 그리드 탐색 · 평가 프로토콜 · Reacher/Ant · 4항 장부
- **13.3** 가우시안 정책 · Reacher 10차원 상태의 내면 · PPO 100만 스텝

13.1 로봇공학 기초와 Sim-to-Real

장난감 환경에서 실체 로봇으로: 3대 기둥

- Week 9–12의 환경(CartPole, Pendulum)은 **단순화된** 로봇 제어 문제.
- 실제 로봇: 팔 · 다리 · 바퀴가 달린 **물리적 시스템** — 행동은 **연속적인 힘/토크**, 물리 법칙(관성 · 중력 · 마찰)을 거쳐 움직인다.

기둥	내용
상태공간	각 관절의 **각도** + **각속도**. 2관절 팔: $(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$ 4차원
운동학	**정기구학**: 관절각 $\rightarrow$ 팔 끝 위치. **역기구학**: 그 반대
동역학	미분방정식: 토크 $\rightarrow$ 가속. 물리 법칙이 등장하는 곳

강화학습 관점에서 중요한 것 하나

MDP의 $\mathcal{S}, \mathcal{A}$가 추상적 기호가 아니라 **실제 각도·힘의 단위를 갖는 연속값**. **Week 9–12의 알고리즘을 물리 법칙이 지배하는 연속 행동공간에 어떻게 적용하는가?**

정기구학: 중학 삼각비로 손 계산

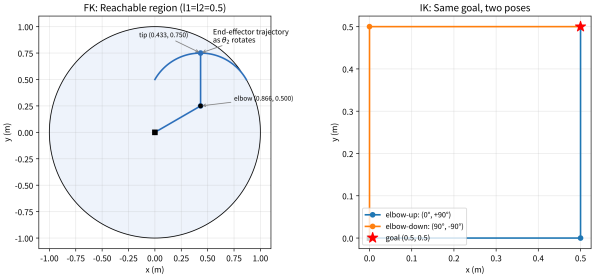
팔 마디 길이 l_1, l_2. 첫 마디가 수평에서 θ_1, 두 번째 마디가 **첫 마디 기준**으로 θ_2 만큼 (절대 각이 아닌 **상대 각**이 로봇공학 관행).

$$x = l_1 \cos\theta_1 + l_2 \cos(\theta_1 + \theta_2), \qquad y = l_1 \sin\theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

- 두 마디의 기여를 그냥 더하면 끝 — 중학교 삼각비.
- 왜 상대 각인가: $\theta_1 + \theta_2$ 가 두 번째 마디의 절대 각. 두 번째 마디는 첫 마디에 "탄" 것이므로, 첫 마디가 도면 함께 도는 효과가 $+\theta_1$ 항으로 나타난다.

숫자 확인: $l_1 = l_2 = 0.5$ m, $\theta_1 = 30^\circ, \theta_2 = 60^\circ$ 라면 팔 마디(elbow) $(0.866,\ 0.500)$, 팔 끝(fingertip) $(0.433,\ 0.750)$.

역기구학: 코사인 정리로 풀기



왼쪽: 도달 가능 영역(반경 1 원판)과 FK 예시. 오른쪽: 목표 $(0.5, 0.5)$ 의 IK 두 해 (elbow-up/down).

팔 끝이 (x, y) 에 있으려면, 팔 끝까지의 거리 $d = \sqrt{x^2 + y^2}$ 가 두 마디와 팔 마디를 이은 "삼각형"의 세 번째 변. 코사인 정리에

$$\cos\theta_2 = \frac{d^2 - l_1^2 - l_2^2}{2 l_1 l_2}$$

- **해가 여러 개**: θ_2 는 arccos 로 부호($\pm$)가 자유로움.
- **도달 가능 영역**: $|l_1 - l_2| \le d \le l_1 + l_2$.

“해가 여러 개”: 같은 목표, 두 갈래

$\theta_2 = \arccos(\cdot)$ 의 부호($\pm$)가 자유롭고, 각 부호마다 다른 θ_1 이 나온다.

$(0.5, 0.5)$ 의 예: $d = \sqrt{0.5}$, $\cos\theta_2 = 0$ 즉 $\theta_2 = \pm 90^\circ$. 두 해는

$$(\theta_1, \theta_2) = (0^\circ, +90^\circ) \text{ (elbow-up)}, \qquad (90^\circ, -90^\circ) \text{ (elbow-down)}$$

서로 90도씩 “거울”처럼 다른 관절 각으로 **같은 팔 끝 위치**를 만듦.

강화학습과의 연결

같은 목표(팔 끝 위치)를 향한 “좋은” 관절 궤적이 **두 갈래**(그 이상) 라는 뜻. 정책이 상태 전체(관절각 + 각속도)를 보게 하는 이유가 여기 — 목표 위치 만 보고 행동하면 어느 갈래로 갈지 운에 맡기는 셈. (13.3절에서 이 “여러 갈래의 행동”을 배우는 PPO 구조를 봄.)

“도달 가능 영역”: 팔이 못 닿는 곳

코사인 값이 $[-1, 1]$ 에 있으려면

$$|l_1 - l_2| \le d \le l_1 + l_2$$

즉 **도달 가능 영역**(reachable workspace)은 반지름 $l_1 + l_2$ 와 $|l_1 - l_2|$ 인 **환**(또는 원, $l_1 = l_2$ 면 중심까지).

- $l_1 = l_2 = 0.5$ 면 $d \le 1.0$ — 팔 끝은 원점 반경 1m 원판 **밖에 절대 못 감**.
- $(1.0, 0.0)$ 처럼 가장 먼 점: 두 해가 $(0°, 0°)$ 하나로 **중첩**(팔이 쭉 펴진 자세) — “해가 여러 개”가 “해가 하나”로 수렴하는 경계점.

이 개념은 13.3절 PPO 실습의 Reacher 과제가 목표 지점을 무작위로 놓는 설계의 열쇠 — 너무 먼 목표면 과제 자체가 불가능해진다.

동역학: 위치에서 운동으로 — 진자의 방정식

기하학은 "로봇이 어딘가에 있다"만 말할 뿐, "힘을 주면 어떻게 움직이는가"는 못 함. 후자가 **동역학**(dynamics). MDP 전이함수 $P(s' \mid s, a)$ 가 "시뮬레이션 한 스텝"이라면, 그 한 스텝의 내용이 바로 동역학 방정식.

가장 단순한 예 **진자**: θ 는 세로(위)에서 잰 각도 (0 이 위쪽 수직 = **불안정한** 균형점, $\pm\pi$ 가 아래 매달린 **안정**점). 균일한 막대(m, l, $I = \frac{1}{3}ml^2$)에 뉴턴-오일러 방정식(중력 토크 $\frac{1}{2}mgl\sin\theta$):

$$\tfrac{1}{3}ml^2\,\ddot{\theta} = \tfrac{1}{2}mgl\sin\theta + u \quad\Longrightarrow\quad \ddot{\theta} = \frac{3g}{2l}\sin\theta + \frac{3}{ml^2}\,u$$

u = 우리가 주는 토크(**행동!**).

- (1) $\sin\theta$ 항 = 위쪽 수직에서 **떨어지려 하는 힘** — 0(위) 근처에서 $\ddot{\theta} > 0$, 더 벌어진 쪽으로 가속. 즉 0은 불안정한 균형(진자를 세우기 어려운 이유).
- (2) u 의 계수 $3/(ml^2)$: 무거울수록($m\uparrow$), 길수록($l\uparrow$) 같은 토크가 작은 가속을 준다 — **관성의 법칙** 그 자체.

진자 식이 코드가 되는 순간 — 시뮬레이션의 정체

Gymnasium의 Pendulum-v1(13.2절 실습의 주인공)이 내부에서 정확히 이 식을 쓴다: $g = 10, m = l = 1$ 로 대입하면 $\ddot{\theta} = 15 \sin\theta + 3u$, 이를 **오일러 적분**으로 스텝:

```
# Pendulum-의v1 step() 핵심 (g=10, m=1, l=1, dt=0.05, max_speed=8)
new_thdot = thdot + (15.0 * np.sin(th) + 3.0 * u) * dt
new_thdot = np.clip(new_thdot, -8.0, 8.0)   # 속도상한
new_th    = th + new_thdot * dt             # 반암시(-) 오일러적분
```

“반-암시(semi-implicit)” — 새로 계산한 각속도로 각도를 갱신하는, 안정성이 좋은 적분 방식.

“시뮬레이션”의 정체

미분방정식을 작은 간격 $dt = 0.05$ s 로 **끊어 반복**. 200스텝 에피소드 = **실제 시간 10초**. “물리 엔진”이라 불리는 이유.

관절이 2개 이상이면 — 동역학의 일반형

$$M(\theta)\,\ddot{\theta} + h(\theta, \dot{\theta}) = \tau$$

- $M(\theta)$ = **관성 행렬**: "지금 자세에서 각 관절에 토크를 주면 얼마나 가속되는가"를 담은 $n \times n$ 행렬. **자세에 따라 바뀜** (팔이 쭉 펴졌을 때와 접혔을 때의 관성은 다름).
- h = 원심력 · 코리올리력 · 중력 합.
- 2관절 팔: $M(\theta)$ 에 $\cos\theta_2$ 같은 항이 등장 — "행동 효과"가 "현재 상태"와 곱해져서 나온다.

핵심 요지 한 줄

동일한 토크 τ(행동)가 자세 θ(상태)에 따라 **다른 가속**을 만든다 — 로봇의 전이는 비선형이고 상태 의존. (10.3절의 Critic이 상태 의존적 $V(s)$ 를 배워야 하는 물리적 이유이기도 함.)

PD 제어: 줄 + 댐퍼, 그리고 $k_p > 5$ 조건

동역학이 "힘 $\rightarrow$ 운동" 규칙을 주면, 남은 문제는 "어떤 힘 u 를 줄 것인가" — 바로 정책 $\pi(a \mid s)$. 사람이 직접 설계하는 가장 유명한 정책이 **PD(Proportional-Derivative) 제어기**:

$$u = -k_p\,(\theta - \theta^\star) - k_d\,\dot{\theta}$$

- **비례 항** $-k_p \Delta\theta$ = 벗어날수록 세게 당기는 줄(스프링).
- **미분 항** $-k_d \dot{\theta}$ = 움직임을 말려 붙이는 댐퍼.

진자(위쪽 $\theta^\star = 0$ 세우기)에 대입하면 $\ddot{\theta} = (15 - 3k_p)\theta - 3k_d\dot{\theta} + \cdots$ 로 선형화. **균형이 잡히려면** $3k_p > 15$**, 즉** $k_p > 5$ — 중력의 "역줄" (위에서 밀어내는 15θ 항)보다 제어기의 줄이 더 강해야.

- 실습 그리드: $k_p = 8$ 인 PD가 진자를 **세운 채** 유지, $k_p = 4$ 는 그냥 **흔드는 데** 그침 (13.2절에서 실험적으로 확인).
- 13.2절 코드 `action = -2.0 * theta - 0.5 * thdot` 도 같은 형태의 PD (그 예는 50스텝 보상을 보느라 "세우는" 성능까지 평가 안 함).

PD가 있는데 왜 강화학습? 그리고 sim-to-real

PD가 있는데 왜 RL이 필요한가?

1. **계수는 "이 진자"에 대해 사람이 수학으로 골라낸 것**. 마찰이 생기거나 질량이 바뀌면 다시 유도해야. RL은 그 유도를 **데이터**로 대체.
2. **PD는 "목표까지의 직선 경로"밖에 모름**. RL은 보상 함수가 주는 어떤 목표든 같은 기계로 접근.
3. **가장 중요한 전제**: PD가 잘되는 곳에는 PD를 쓰는 것이 맞다. RL이 PD를 이겨야 하는 곳은 "사람이 식을 세우기 어려운" 곳.

실물 로봇으로 **직접** 학습하면: 로봇 **고장** · 사람에게 **위험** · 무엇보다 **느림**. 그래서 실전은 **시뮬레이션**에서 학습 → **실물로 전이** (**sim-to-real**).

실전 구조 + 남은 질문

PD를 **베이스라인**으로 두고 RL이 넘었는지 확인 (13.2절). **시뮬레이션이 실제 물리를 완벽히 재현하지 못한다면 이 파이프라인은 어떻게 믿을 것인가?**

현실 격차: 요인과 도메인 무작위화

시뮬레이션은 실제 물리를 완벽히 재현 못 함. 시뮬레이션에서만 잘 작동하고 실물에서 실패하는 정책 = **현실 격차**(reality gap).

요인	시뮬레이션 vs 실물
마찰 · 감쇠	단순 모델 vs 줄 경직 · 기어 마찰 · 볼 베어링 손실 복합
액추에이터 지연	즉시 출력 vs ms 단위 지연 + 데드존
센서 잡음 · 지연	— vs 인코더 분해능, IMU 드리프트
접촉 모델링	“점 접촉” vs 표면 재질 · 탄성
질량 분포	설계값 vs 생산 편차, 배터리 잔량

대응 1: (domain randomization, Tobin et al., 2017)

물리 파라미터(마찰, 질량, 조명 등)를 **매 학습 에피소드마다 무작위로 바꿔** 학습 → “다양한 물리 조건 전체”에 잘 동작하도록 강제. Week 6(ML1)의 정규화와 구조가 같음.

“노이즈”가 아니라 “시스템틱한 편차”

- 예: 시뮬레이션의 마찰 계수가 실제보다 **3배** 작다면, 시뮬레이션 에피소드 백만 개를 모아도 그 오차는 **사라지지 않는다**.
- Week 4(ML1)의 bias-variance 분해로 말하면:

variance는 데이터로 줄이고, **bias**는 데이터로는 줄이지 못한다.

- bias는 “모델을 더 잘 맞추면” 줄일 수 있을 뿐.
- 그래서 시뮬레이션에서 더 많이 학습하는 것만으로 격차가 해결되지 않는다 — 격차의 원인을 **구조적으로** 다루는 방법 (무작위화, 모델 보정)이 필요.

숫자로 체감하기 — 20개 PD 정책 실험

`Pendulum-v1` 동역학 식을 직접 코드로 재현한 뒤, 감쇠 계수 μ를 "시뮬레이션 $\mu = 0.1$, 실물 $\mu = 5.0$"로 가정하고(이 정도 벌어지는 경우가 흔함), 20개 PD 정책($k_p \in \{1, 2, 4, 8\}$, $k_d \in \{0, 0.5, 1, 2, 4\}$, 각 12 시드)의 200스텝 누적 보상을 두 "환경"에서 측정:

기준	최고 점수	그 정책
시뮬레이션($\mu = 0.1$)에서 최고	-1163.9	$k_p = 8, k_d = 2$
실물($\mu = 5.0$)에서 최고	-1204.3	$k_p = 8, k_d \in \{0, 0.5, 1, 2\}$
시뮬레이션 20개 정책의 범위	$[-1463.0,\ -1163.9]$	—
실물 20개 정책의 범위	$[-1422.3,\ -1204.3]$	—

여기서 "왜?"를 **세 단계**로 읽는다.

세 단계로 읽기: 편이는 구조다, 순위는 "대체로"만

(1) 정책마다 다른 양으로 벌어진다

- 고정된 오프셋이면 차를 빼면 그만. 그런데 실물 − 시뮬레이션 차는 **정책마다** 달라짐:

  $(k_p=1, k_d=0)$: -67.0 $(k_p=4, k_d=4)$: $+103.9$ $(k_p=8, k_d=2)$: -40.4

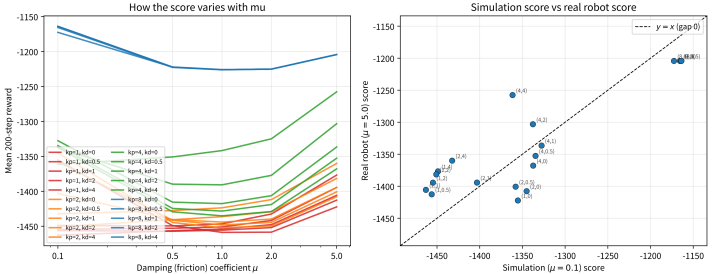
- 전체 평균은 $+5.0$에 불과하나 정책별 차이는 약 **170pt** 차이 — "운"이 아니라 "마찰 모델"이라는 **구조**. 데이터를 **늘려도 안 사라짐**.

(2) 그래도 순위는 대체로 유지: 시뮬레이션 vs 실물 순위 상관계수 = **0.76**. 0.05였다면 시뮬레이션 학습은 전혀 쓸모없었을 것 — "약하게나마" 실물과 상관하는 것이 sim-to-real의 존재 근거.

(3) 그러나 "대체로"가 문제: $(k_p=1, k_d=0)$는 시뮬레이션 -1355.3 = 20개 중 **11위**(상위권) → 실물 -1422.3 = **20위(꼴찌)**.

시뮬레이션 하나로 정책 하나를 골라 "완료"라고 선언하는 순간,
그 순간부터 격차가 실수로 바뀐다.

현실 격차의 시각화



왼쪽: 20개 PD 정책을 감쇠 $\mu \in \{0.1, 1, 2, 5\}$ 환경에서 실행한 200스텝 누적 보상 곡선 (색 = k_p). 오른쪽: 시뮬레이션($\mu = 0.1$) vs 실물($\mu = 5.0$) 산점도.

- μ 가 커질수록 점수는 낮아지되, $k_p = 8$ 계열이 가장 **늦게** 떨어짐.
- $\mu = 5.0$ 에서는 k_d 의 차이가 거의 사라짐.
- 산점도는 $y = x$ 대각선 주변에 모이나, **정책마다 다른 편이**를 보임 — 이 편이가 바로 "현실 격차".

도메인 무작위화: 범위를 무작위화하라, 최악을 최소화하라

"시뮬레이션 점수만 보고 고르는 것"의 함정: 실물의 μ 를 정확히 알 수 없으니까.

도메인 무작위화는 이 문제를 학습 중에 푼다: μ 를 매 에피소드마다 $\{0.1, 1, 2, 5\}$ 처럼 범위에서 무작위로 뽑아 훈련 $\rightarrow$ "$\mu = 0.1$ 인 세계"에 과적합할 기회 자체가 사라짐.

시뮬레이션 최고점 정책 $(k_p{=}8, k_d{=}2)$ 의 μ 별 점수: 0.1: -1163.9 1.0: -1225.9 2.0: -1225.1 5.0: -1204.3

가장 위험한 환경은 예측하지 못한 곳

시뮬레이션(0.1)도, 가정한 실물(5.0)도 아닌 $\mu = 1.0 \sim 2.0$ **이라는 중간 값**. $\rightarrow$ 어느 μ 에서도 나쁘지 않은 정책 = **"모든 μ 에서 가장 늦게 떨어지는" ($k_p = 8$ 계열)** — 이 절의 모든 이야기가 이 한 줄로 요약됨.
정규화 강도 λ 와 정확히 같은 **트레이드오프**.

무작위화 범위: 너무 크면 과제를 스스로 깨뜨린다

- 예: $\mu = 100$ 처럼 **극단적 마찰**까지 포함하면, 그 극단에 최적화한 정책이 실물 $\mu = 5$ 에서 **오히려 나빠짐**.
- “극단 조건에서 잘하는 법”을 배운 대가로 “보통 조건에서 잘하는 법”을 망각하는 — **과적합의 거울상**.

따라서

무작위화 범위는 **실물 로봇의 “최악의 경우”가 어느 정도인지에 대한 공학적 판단**을 코드로 번역한 하이퍼파라미터.

시스템 ID: 실물의 “참 파라미터”를 재다 — 그리고 한계

“시뮬레이션을 실물에 맞게 고치면 되지 않나?” — 바로 그 방향이 **시스템 식별**(system identification): 실물에 알려진 토크 패턴을 주고 응답(각도·각속도)을 측정, 그 데이터에 시뮬레이션 모델을 **피팅**해 μ(질량·관성 등)를 추정.

- **무작위화와 보완 관계**: 측정으로 정밀화 = 무작위화 범위를 좁히는 방향. 측정한 파라미터는 범위에서 빼고, 못 측정한 나머지만 무작위화.
- **ID만으로 안 사라지는 이유**: (1) 측정 못한 상태는 여전히 미지(학습 중 처음 겪는 자세), (2) 격차는 “단일 계수”로 못 박을 수 없는 구조적 차이 (액추에이터 지연, 접촉 비선형), (3) 로봇은 마모한다(동역학이 천천히 바뀜).

실전 표준

시스템 ID + 무작위화 + 실물 소량 데이터 fine-tuning을 **조합** — “측정” + “범위” + “보정”.

역사적 일화: 바나나를 뒤집은 손, MuJoCo의 10년

"sim-to-real이 통하는가"는 2010년대 내내 열린 질문. 답은 두 차례의 시범으로 굳어졌다.

- **(1) MuJoCo**(Todorov et al., 2012): 2012년 개발 시작, 2015년 공개, 2018년 무료 오픈소스. 접촉·마찰 모델링이 정밀하고 빠름 → 2018년 이후 로봇 RL 논문의 **표준 물리 엔진**.
- **(2) OpenAI의 바나나 뒤집기(2021)**: 10개 손가락 근손(dexterous hand)이 시뮬레이션에서 수백만 에피소드 학습 → 실물 바나나를 집어 뒤집는 시연. 결정적: 격차를 도메인 무작위화로 메운 뒤(조명·질감·마찰), 실물 데이터 소량만 fine-tune.
- 이후: 2020년 OpenAI **A1 로봇**("10분 만에 새로운 걸음걸이 학습", Rudin et al., 2021), 2020년대 후반 휴머노이드까지.

파이프라인

"시뮬레이션에서 배우고 → 실물로 옮긴다"가 로봇 학습의 **기본 구조**가 됨 — 이번 장(13∼15장)이 그 기초를 쌓는 이유.

13.1 자주 하는 실수

1. **“시뮬레이션에서 검증했다 = 완료”로 선언**. 시뮬레이션 점수는 시뮬레이션 환경의 점수일 뿐 — 시뮬레이션 하위권 정책이 실물에서 상위권으로 반전될 수 있음. 배포 전 실물에서(소량이라도) 평가하는 단계가 **반드시** 필요.
2. **“시뮬레이션은 무한히 빠르니까 데이터 부족은 없는 문제”**. 학습은 사실상 무한 데이터지만 배포는 실물에서. “시뮬레이션 1천만 에피소드”와 “실물 0시간”은 다른 자원.
3. **무작위화 범위를 “수학적으로 안전한” 극단까지 확장**. $\mu = 100$ 예처럼 실물에서 불가능한 조건까지 포함하면 정책이 그 극단에 최적화되어 실물에서 오히려 나빠짐. “넓을수록 안전”이 아님.
4. **PD의 “마법 수”를 RL로 대체할 수 있다고 생각**. $k_p > 5$ 는 이 진자의 물리에서 유도되는 한계. RL이 이 한계를 위배하는 방향으로 학습하면 **학습 설정이 잘못됐다**는 진단 신호. 물리 직관은 RL의 대체재가 아니라 **감시자**.

13.1 핵심 정리

핵심 정리

로봇의 상태(각도·각속도)와 행동(토크)는 **물리적 단위를 갖는 연속값**. **정규구학/역기구학**이 "각도 $\leftrightarrow$ 위치"를, **동역학** $M(\theta)\ddot{\theta} + h = \tau$ 가 "토크 $\leftrightarrow$ 가속"을 계산. 시뮬레이션은 이 식을 dt 로 끊어 **반복**하는 것뿐.

현실 격차는 시뮬레이션과 실물의 파라미터·구조 차이가 만든 편이이므로 **데이터로 줄이지 못하고**, **도메인 무작위화**(환경의 다양화)와 **시스템 ID**(파라미터 측정)로 줄인다. 무작위화 범위는 정규화 강도와 같은 트레이드오프 하이퍼파라미터이며, "시뮬레이션 최고점"은 "실물 최적"을 함의하지 **않는다**.

13.2 로봇 환경 탐색 실습

실습: Pendulum-v1 + 손으로 만든 PD 제어기

Gymnasium의 로봇류 환경. CartPole·Pendulum도 사실 가장 작은 형태의 관절 제어 문제.
Pendulum-v1: 상태 $(\cos\theta, \sin\theta, \dot{\theta})$, **연속** 행동공간 Box$(-2.0, 2.0)$ (토크).

```
obs, info = env.reset(seed=0)
total_reward = 0.0
for _ in range(50):
    cos_th, sin_th, thdot = obs
    theta = np.arctan2(sin_th, cos_th)
    # 손으로설계한  PD비례미분(-) 제어기
    action = np.clip(-2.0 * theta - 0.5 * thdot, -2.0, 2.0)
    obs, reward, terminated, truncated, info = env.step([action])
    total_reward += reward
print("PD 제어기의스텝 50 누적보상 :", round(total_reward, 2))
```

이 PD 제어기는 강화학습이 전혀 아니다 — 사람이 직접 설계한 규칙. 그런데도 무작위보다 낫다. 그러나 "나은가"를 판정하려면 **평가 프로토콜**이 먼저다.

50스텝으로는 구분이 애매 — 창을 같게 늘려야

같은 코드로 시드 0∼3 를 돌려 50스텝 누적 보상:

- **무작위** 정책: $-234 \sim -403$
- 이 PD 규칙($k_p = 2, k_d = 0.5$): $-157 \sim -382$

50스텝 창에서는 두 정책의 구분이 애매. 평가 창을 200스텝 (실시간 10초)으로 같이 늘리면 둘이 갈라진다:

$k_p = 8, k_d = 2$ 의 PD: ≈ -455 (3시드 평균)　　무작위(같은 프로토콜): 평균 ≈ -983

이 비교가 보여주는 것

강화학습의 목표 = 이런 손으로 설계한 제어기보다 더 나은, 또는 사람이 규칙으로 설계하기 어려운 상황에서도 작동하는 정책을 **데이터로부터 직접 찾아내는 것**.

평가 프로토콜: 왜 50스텝으로는 안 되는가

1. **시작 자세가 랜덤** — `Pendulum-v1` 의 `reset` 이 초기 각도를 균분포에서 뽑으므로 매 에피소드가 다른 문제. 문제 하나만 내고 점수를 보면, 점수의 대부분이 "어떤 문제가 나왔는가"라는 운에 지배.
2. **진자의 목표는 "흐름"이 아니라 "상태"** — "위로 세운 채 유지"는 장기적 성취. 50스텝(2.5초) 창은 10초 에피소드의 첫 순간 이동만 측정 — 초기 이동(첫 휘청임)이 짧은 창을 지배하고 장기 성능은 드러나지 않음.

실습 규칙: 창 고정 · 다중 시드 · 창을 뒤돌아 바꾸지 마라

이 절(과 13.3절)의 실습 규칙

1. 정책을 평가할 때 **창을 고정**(여기선 200스텝).
2. **시드 여러 개**(여기선 3개)의 평균을 본다.
3. **결과를 보고 나서 창을 바꾸지 않는다**.

세 번째가 **함정** — "50스텝 숫자가 보기 싫어서 200스텝으로 해보자"는 것은 평가가 아니라 **평가 프로토콜의 하이퍼파라미터 튜닝**.

전형적인 실수: 다른 창 숫자를 한 비교에 섞기. "무작위 200스텝 (≈ -983)"과 "PD 50스텝(≈ -263)"을 대면 PD가 "4배 좋다"지만, 그건 정책의 차이가 아니라 **창의 차이**.

평가 프로토콜도 실험의 일부다 — 비교하기 전에 창·시드 수·초기 분포가 같은지부터 확인하라.

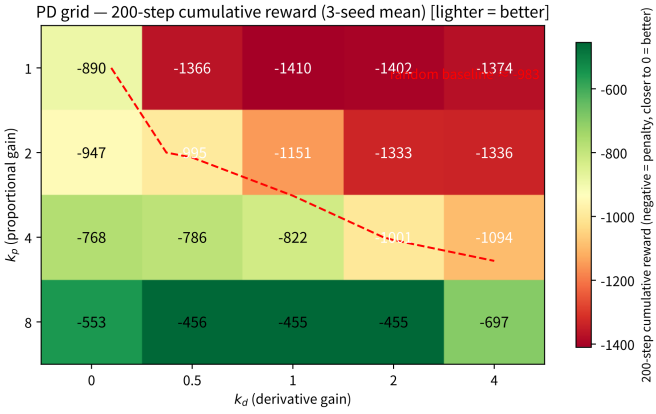
PD 그리드: 20개 “손으로 만든 정책”을 훑다

PD 제어기는 노브가 딱 두 개 (k_p, k_d). $k_p \in \{1, 2, 4, 8\}$, $k_d \in \{0, 0.5, 1, 2, 4\}$ 로 20개 정책을 만들고, 각각 200스텝 $\times$ 3시드 평균으로 누적 보상 측정:

	$k_d = 0$	$k_d = 0.5$	$k_d = 1$	$k_d = 2$	$k_d = 4$
$k_p = 1$	-889.7	-1365.8	-1410.1	-1401.8	-1374.2
$k_p = 2$	-947.3	-995.4	-1150.8	-1333.1	-1336.2
$k_p = 4$	-767.9	-785.8	-821.7	-1001.3	-1094.0
$k_p = 8$	-553.3	-455.7	-455.0	-454.9	-697.0

무작위 정책 같은 프로토콜(200스텝 $\times$ 3시드) 평균: ≈ -983.

PD 그리드 히트맵: 행은 클수록 낫고, 열은 아니다



20개 조합의 200스텝 누적 보상 (3시드 평균). 진자 보상은 벌(음수)이라 밝을수록 좋음. 빨간 점선 = 무작위 기준 ≈ -983.

- **행(k_p): 클수록 낫다** — 13.1절의 물리 그대로. $k_p = 8$ 행($-455 \sim -553$)이 $k_p = 4$ 행 ($-768 \sim -1094$) 압도.
- **열(k_d): 클수록 좋은 게 아니다** — $k_d = 4$ 열은 댐퍼가 너무 세어 움직임 자체가 말려 붙어 "느리게 실패".
- **최선 $(k_p, k_d) = (8, 2)$: ≈ -455** — 무작위 ≈ -983 대비 누적 벌이 **2.2배**로 감소.

“그리드 방식은 고차원에서 죽는다” — RL의 직접적 동기

이 그리드 탐색은 **정책 탐색의 본질**: 20개 정책 = 2차원 파라미터 공간의 20개 점을 사람(스크립트)이 하나하나 훑은 것.

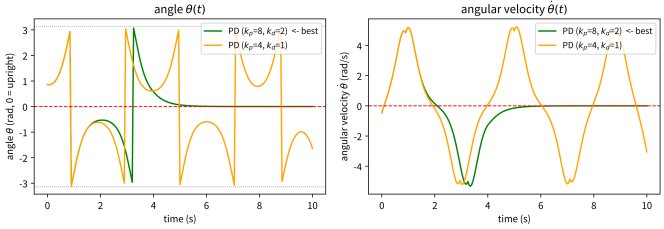
그런데 Ant에 같은 방식은 불가능:

- Ant의 행동은 **독립 8차원**, “이득 $\times$ 오차” 형태로 “좋은 정책”을 두 숫자로 못 박는 형태가 존재하지 않음.
- 정책은 **105차원 상태 $\rightarrow$ 8차원 행동의 함수**, 그 파라미터 공간은 1만 개가 넘는 가중치의 공간 (64-64 은층을 그대로 쓰면 $105 \times 64 + 64 \times 64 + 64 \times 8 \approx 11500$ 개).

그리드 방식은 고차원에서 죽는다 — 이것이 “정책을 데이터로부터 배운다”는 강화학습의 직접적 동기.

“좋은 점수” $\neq$ “과제 달성”: 궤적을 보라

점수 표는 마지막 질문에 답 못 함: 최선 $(8, 2)$ PD가 진자를 정말로 “세워 유지”하는가, “잘 흔들고만” 있는가?



왼쪽: 각도 $\theta(t)$, 오른쪽: 각속도 $\dot{\theta}(t)$. 초록 $(8, 2)$ vs 주황 $(4, 1)$. 시드 0, 각 200스텝(10초).

- 초록 $(8, 2)$: 약 5초에 $\theta \approx 0, \dot{\theta} \approx 0$ (위쪽 균형점)에 **수렴해 그 자리에 머묾** — 과제 달성.
- 주황 $(4, 1)$: 위쪽 근처에 $|\theta| < 0.3$으로 **한 번도** 머무르지 못하고 $\pm\pi$ (아래) 중심으로 $|\dot{\theta}| \approx 5$ rad/s까지 10초 내내 “그냥 흔들림”.

왜 궤적을 보나: 누적 보상은 프록시

13.1절이 수학으로 미리 말한 바로 그 사실 ($k_p < 5$ 면 제어기의 "줄"이 중력 "역줄"을 이기지 못해 위쪽 균형이 불안정)을, 여기선 실험으로 확인한 셈.

왜 "궤적을 보는" 단계가 필요한가

누적 보상은 진짜 과제의 프록시(proxy)이기 때문. "누적 벌 최소화"와 "위쪽을 세운 채 유지"는 대부분 같은 방향을 가리키지만 항상 같은 것은 아님. 프록시를 최적화한 뒤 진짜 목표를 한 번 확인하는 습관이 없으면 "점수는 좋은데 과제를 못 한" 정책을 배포하게 됨 — Week 12의 **리워드 해킹** 방어선이 1관절 진자에서부터 시작.

로봇팔·사족보행 환경 탐색하기

Pendulum이 1관절의 가장 단순한 문제라면, 이제 **관절이 여러 개**인 진짜 로봇 환경.
Gymnasium은 MuJoCo 물리엔진 기반 로봇 환경을 제공:

- **Reacher-v5**: 2관절 로봇팔이 **목표 지점에 끝을 맞추는** 과제.
- **Ant-v5**: 다리 4개짜리 사족보행 로봇이 **앞으로 걷는** 과제.

`Reacher-v5` 상태 (10,) 행동 (2,) $[-1, 1]^2$ `Ant-v5` 상태 (105,) 행동 (8,) $[-1, 1]^8$

CartPole(상태 4, 행동 2선택)·Pendulum(상태 3, 행동 1)과 비교하면 **상태·행동공간이 훨씬 크고 복잡**.

무작위 정책을 돌려보며 “분위기”를 잡기

```
for name in ["Reacher-v5", "Ant-v5"]:
    env = gym.make(name)
    obs, info = env.reset(seed=0)
    total_reward = 0.0
    for _ in range(100):
        a = env.action_space.sample()  # 아직학습전이므로무작위
        obs, r, term, trunc, info = env.step(a)
        total_reward += r
        if term or trunc:
            break
    print(f" 무작위정책스텝   100 누적보상 : {total_reward:.2f}")
    env.close()
```

단일 시드 실행 결과: Reacher -41.60, Ant -4.07 (100스텝 누적).

단일 시드는 재현이 안 된다 — 시드된 RNG로

"로봇 환경의 분위기"를 잡는 데는 이 정도면 충분. 그런데 단일 시드의 숫자는 **재현이 안 된다** — `env.action_space.sample()` 가 전역 numpy RNG에서 뽑기 때문에, 이전까지 몇 번 샘플링했는지에 따라 같은 코드도 숫자가 달라진다.

노트북의 교훈: 비교할 숫자는 재현 가능한 프로토콜로 재야 한다

에피소드마다 시드된 RNG(`default_rng(1000+seed)`)로 무작위 행동을 뽑아 시드 0∼2 확장:

환경	시드별 100스텝 리턴
Reacher-v5 (50스텝에서 truncated)	seed0 -42.98, seed1 -40.02, seed2 -39.75
Ant-v5 (조기 종료)	seed0 -6.88 (63스텝), seed1 -19.47 (24스텝), seed2 -9.94 (11

언제 다시 돌려도 같은 숫자. Ant 에피소드가 100스텝 전에 조기 종료된다는 점에 주목 — 다음에서 "장부"로 설명.

Ant의 양수 리턴 에피소드: 보상은 “장부”다

시드 0∼2 는 모두 음수지만, 무작위 Ant 에피소드 중에는 **리턴이 양수**인 것이 있다 (시드 4: $+3.60$, 16스텝에서 `terminated`). “무작위 로봇이 양수 점수를 받았 다니?” — Ant의 보상은 단일 항이 아니라 **4항의 장부**:

$$r = \underbrace{1.0 \times \mathbb{1}[\text{healthy}]}_{\text{생존}} + \underbrace{1.0 \times \dot{x}}_{\text{전진}} - \underbrace{0.5\,\|a\|^2}_{\text{제어비용}} - \underbrace{5 \times 10^{-4} \sum F^2_{\text{contact}}}_{\text{접촉비용}}$$

$\dot{x}$ = 몸통 x 방향 속도(스텝당 0.05 s, `frame_skip = 5`), healthy = 몸통 높이 $[0.2, 1.0]$ m 범위 안. seed4 장부: 생존 $+15$ + 전진 $\approx +10.5$ − 제어 ≈ -21.9 − 접촉 ≈ -0.01 $\approx +3.6$.

“양수 점수”가 “잘 걷는다”는 뜻이 아니다

- “양수 점수” = 15스텝의 생존점 + 짧은 전진이 제어 비용을 비로소 상회한 것일 뿐 — 직관적으로도 “16스텝 만에 넘어졌다”는 에피소드.
- **에피소드 길이 자체가 신호**: seed4는 100스텝이 아니라 16스텝에서 끝남. `terminated`(진짜 “사망”: 몸통 높이 $z < 0.2$ m)이지, Reacher의 50스텝 `truncated`(시간 제한)가 아님.

Week 3의 5-튜플 API가 둘을 나눈 이유

“16스텝에 $+3.60$”(seed4)과 “63스텝에 -6.88”(seed0)을 리턴 평균 하나로만 비교하면, 먼저 넘어진 에피소드가 오히려 “좋은” 것으로 읽힐 수 있음. 로봇 환경 평가: **리턴 + 에피소드 길이 + 종료 비율** 을 항상 함께 보고.

105차원은 무엇을 읽는가: 13 + 14 + 78

Ant의 상태 105차원을 분해하면 **13 + 14 + 78**:

- **13** 개의 위치 좌표 (원점 x, y 는 제외: 몸통 높이 · 자세 각 4 + 다리 관절 8).
- **14** 개의 속도 좌표.
- **78** 개의 **접촉력 측정치** ($cfrc_ext$) — 마지막 항이 성인. 발이 땅을 짚는 감각이 상태의 3분의 2를 차지.

이 덕분에 보행이 "학습 가능"해진다: 정책이 발자국마다의 피드백을 직접 **볼** 수 있기 때문. Reacher는 착지할 필요가 없으니 접촉력 0차원, 상태 10차원에 불과.

상태 차원 = 로봇이 "읽는" 양, 행동 차원 = 로봇이 "쓰는" 양

Ant 105:8은 "많이 읽고 적게 쓰는" 구조 — 균형을 위해 읽어야 할 정보가 노브(8개 토크)보다 훨씬 많음. "8개 토크를 어떤 상태에 주목해서 정할 것인가"라는 **주의(attention) 문제**.

“큰 로봇”의 물리적 대가: 스텝당 시간

모델이 크면 스텝당 계산도 무거워진다 (2000회 `env.step()` 루프, 로컬 측정):

환경	상태 차원	행동 차원	스텝당 소요(측정)
Pendulum-v1	3	1	$\approx 23\ \mu$s
Reacher-v5	10	2	$\approx 37\ \mu$s
Ant-v5	105	8	$\approx 180\ \mu$s

- Ant가 Reacher의 약 5배지만, 둘 다 노트북 CPU에서 “학습할 수 있을 만큼” 빠름 — 물리 엔진이 빨라서 학습이 병목이 안 되는 것이 MuJoCo가 연구 표준이 된 이유 (13.1절 역사적 일화).
- 실무에서 병목이 되는 것은 거의 항상 **필요 스텝 수**: 13.3절의 PPO는 Reacher에서 100만 스텝을 써야 1줄 제어기와 어깨를 나란히. 스텝당 시간(5배)은 선형 문제, 필요 스텝 수는 알고리즘 난이도의 본질 문제 (스텝당 시간을 줄이는 GPU 병렬은 Week 14.2).

13.2 자주 하는 실수

1. **비교할 때 평가 창(horizon)을 맞추지 않는 것**. “200스텝 무작위(≈ -983)”와 “50스텝 PD(≈ -263)” 대면 PD가 “4배 좋다”지만 **창의 차이** — 같은 창·시드로 재면 2.2배로 줄어듦. 창이 다르면 차이는 “정책의 차이”가 아니라 “프로토콜의 차이”.
2. **점수만 보고 궤적을 확인하지 않는 것**. $(4, 1)$ 은 -821.7 로 “그저 그렇다” 보이나, 궤적이면 위쪽을 한 번도 세워보지 못했다는 것이 드러남. 누적 보상은 프록시 — 배포 전 진짜 목표를 궤적으로 확인하는 단계는 생략 불가.
3. **조기 종료 환경에서 리턴 평균만 보고 길이·종료 비율을 버리는 것**. Ant seed4의 “$+3.60$”은 16스텝 만에 쓰러진 에피소드. **리턴 평균 + 에피소드 평균 길이 + terminated 비율**을 세 가지를 한 줄로 함께 보고하는 것이 로봇 환경 평가의 최소 문법.
4. **Pendulum의 1줄 제어기를 Ant로 “확장”할 수 있다고 생각**. 진자의 PD는 2개의 숫자였지만 Ant의 정책은 105$\to$8 함수. “왼쪽 앞다리를 뻗는 동안 몸통 높이를 유지하고 …” 같은 조율을 표현할 “이득 $\times$ 오차” 형태가 존재하지 않음 — 13.3절에서 PPO를 쓰는 직접적 이유.

13.2 자주 묻는 질문

- **PyBullet과 MuJoCo 중 뭘 써야 하나?** 둘 다 물리 엔진이고 Gymnasium이 양쪽 지원. MuJoCo는 계산 정확도(특히 접촉·마찰)가 높아 최근 연구에서 더 널리 쓰이고, `pip install gymnasium[mujoco]` 만으로 별도 컴파일 없이 설치. 이번 학기 로봇 실습은 **MuJoCo** 기본 (Week 14에서 엔진 자체를 더 자세히 비교).
- **Reacher와 Ant의 상태·행동공간 크기가 왜 이렇게 다른가? 자유도**의 차이. Reacher는 팔 관절 2개, Ant는 다리 4개(각 2개씩 = 8개)를 동시에 조율 — 몸에 붙은 관절이 많을수록 "동시에 조율해야 할 변수"가 늘어나는 것이 로봇 제어가 어려운 이유.
- **Ant 무작위 리턴이 양수($+3.60$)인 것이 "거의 걸을 줄 안다"는 뜻인가?** 아님 — 16스텝 만에 terminated. "걸을 줄 안다"의 기준은 리턴이 아니라 **에피소드 전체(100스텝)를 유지 하면서 전진 속도 $\dot{x}$ 가 지속 양수**.
- **Reacher는 항상 50스텝(시간 제한)인데 Ant는 16스텝에 끝나기도 — 평가 코드에 영향이?** 아주 작음: `terminated or truncated` 로 break 한 줄이면 같은 루프. 다만 평가 지표가 "고정 100스텝"을 가정하면 안 됨 (16스텝 뒤에 "가상의 84스텝 리턴 0"을 덧붙이는 식의 처리 금지).

13.2 핵심 정리

핵심 정리

로봇 환경 실습의 첫 단계는 알고리즘이 아니라 **평가 프로토콜** — **창 고정, 다중 시드 평균**, 조기 종료 환경에서는 리턴 평균과 함께 **에피소드 길이·종료 비율**까지 보고.

손으로 만든 정책(PD)은 2차원 파라미터 공간에서는 그리드로 탐색 가능 (최선 $k_p = 8, k_d = 2$, ≈ -455 vs 무작위 ≈ -983) 하지만, "105차원 상태 $\rightarrow$ 8차원 행동"의 고차원에서는 **형식적으로 죽는다**.

누적 보상은 진짜 과제의 프록시이므로 좋은 점수를 보면 반드시 **궤적**까지 확인. 보행 로봇의 보상은 **4항 장부**(생존 + 전진 − 제어 − 접촉)이며, "양수 리턴"이 "16스텝 만에 넘어진" 에피소드일 수 있다.

13.3 연속제어환경에 PPO 적용하기

PPO를 연속 행동으로 확장: 가우시안 정책

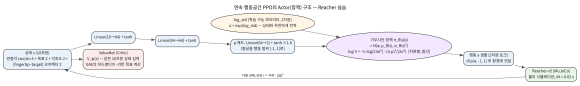
Week 11의 PPO는 이산 행동(softmax)뿐 아니라 **연속 행동**에도 자연스럽게 확장된다 — 신경망이 각 행동 차원마다 정규분포의 평균과 표준편차를 출력하도록 하면 된다:

$$\pi_\theta(a \mid s) = \mathcal{N}(a;\ \mu_\theta(s),\ \sigma_\theta(s)^2)$$

```
import math
def gaussian_log_prob(action, mu, sigma):
    # 연속행동의로그확률     -- softmax 대신정규분포를쓴다는점만다름
    return -0.5 * math.log(2 * math.pi * sigma**2) \
           - (action - mu)**2 / (2 * sigma**2)
```

로그 확률 $\log \pi_\theta(a \mid s)$ 는 정규밀도에서 바로 계산. Week 10.1의 로그미분 트릭과 Week 11.2의 클리핑 목적함수는 이산/연속 여부와 무관하게 형태가 **그대로 유지** — “행동을 어떻게 파라미터화하는가”만 바뀔 뿐 그 뒤의 최적화 기계는 동일.

Actor의 구조: 트렁크 + μ 헤드 + 전역 표준편차



10차원 상태 $\rightarrow$ 64-64 트렁크 $\rightarrow$ μ 헤드(tanh) + 전역 log_std $\rightarrow$ 2차원 가우시안 정책. ValueNet은 $V(s)$ 추정.

세 부분:

- **(1) 트렁크**(64-64 은층): 10차원 상태 처리 — 11.2절 Pendulum Actor와 같은 뼈대, 입력 차원만 바뀜.
- **(2) μ 헤드**: 64차원 특징 $\rightarrow$ 2차원 평균. 출력을 tanh$\times$1.0 — 평균을 행동 범위 $[-1, 1]$ 안에 가둬두면 정책 "중심"이 로봇이 낼 수 있는 범위를 벗어남.
- **(3) log_std 파라미터**: 표준편차. s **가 붙지 않음** — 모든 상태에 공유되는 2차원 벡터 하나.

왜 σ 도 상태 함수가 아닌가

"σ 도 두 번째 헤드에서 $\log \sigma(s)$ 로 출력하면 안 되나?" — 가능하지만, 표준 PPO 구현은 **전역 버전**이 일반적:

- **(1) 파라미터 경제성**: 상태별 σ 는 64→2 출력 헤드가 하나 더 늘지만 이득 제한적 — "얼마나 대담하게 탐색할지"는 대부분 상태의 성질보다 과업 전체의 성질.
- **(2) 안정성**: $\log \sigma$ 는 엔트로피 보상에 직접 들어가는 값(11.2절의 `-0.01 * d.entropy()`). 상태마다 σ 가 들썩이면 엔트로피 항도 들썩여 그래디언트를 불안정하게 만듦.

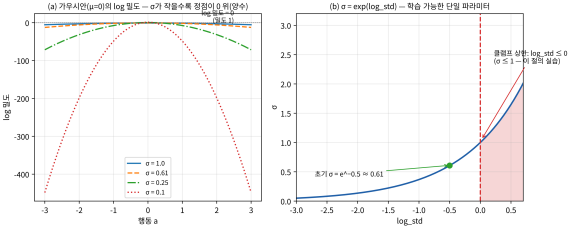
"간단한 것부터, 안 되면서 복잡하게"라는 11.1절 클리핑 하이퍼파라미터 선택의 교훈이 또 한 번 등장.

로그 확률의 함정: 밀도의 로그이므로 양수

이산(10.2절 softmax)에서는 $\pi(a \mid s) \in [0, 1]$ 이라 $\log \pi(a \mid s) \le 0$ 이 항상 성립. 연속에서는 이 직관이 **깨짐**:

- $\pi_\theta(a \mid s)$ 는 확률(確率)이 아니라 **확률밀도** — $[0, 1]$ 로 묶이지 않고 **1보다 커질 수** 있음.
- σ 가 작으면 평균에서의 밀도 $\frac{1}{\sigma\sqrt{2\pi}}$ 가 커짐: $\sigma = 0.25$ 면 1.596, $\sigma = 0.1$ 면 3.99.
- 즉 **로그 확률이 양수**일 수 있음 — `Normal.log_prob` 가 드물게 양수를 돌려주면 버그가 아니라 이 현상.

밀도의 로그가 양수여도 괜찮은 이유 + $\log\sigma$ 를 배우기



(a) $\mu=0$ 가우시안의 log 밀도: σ 가 작을수록 좁고 높고, 정점이 0 위(양수)로. (b) $\log\sigma$ 학습 파라미터, ≤ 0 클램프(빨간 점선). 초기 $\log\sigma = -0.5$.

클리핑 목적함수는 두 밀도의 비만 봄:

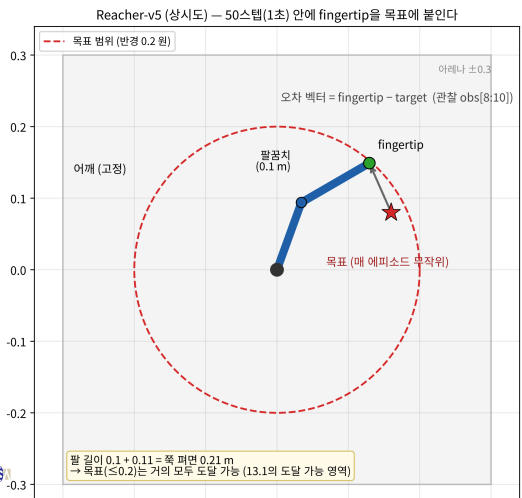
$$r_t = \exp\left(\log\pi_{\text{new}} - \log\pi_{\text{old}}\right)$$

로그 밀도의 차이를 취하는 순간 상수 항 $-\frac{1}{2}\log(2\pi\sigma^2)$ 가 **서로 소거**.

σ 를 직접 배우지 않고 $\log\sigma$ (log_std)를 배움 — $\sigma = \exp(\log\sigma)$ 한 줄이 $\sigma > 0$ 을 자동 보장.

Reacher-v5: “상태 10, 행동 2”의 내면

Reacher-v5: 링크 길이 **0.1 m + 0.11 m**인 2관절 평면 로봇팔 (팔 끝 fingertip는 팔꿈치 기준 0.11 m, 완전히 쭉 펴면 **0.21 m**까지 닿음). 과제: **50스텝(실시간 1초, $dt = 0.02$ s) 안에 fingertip을 무작위 목표 지점에 붙이기**.



10차원 상태 (`reacher_v5.py` 의 `_get_obs`):

인덱스	내용
obs[0:2]	$\cos\theta_1, \cos\theta_2$ (두 관절각 코사인)
obs[2:4]	$\sin\theta_1, \sin\theta_2$ (두 관절각 사인)
obs[4:6]	(x_t, y_t) 목표 지점 좌표 (m)
obs[6:8]	$\dot\theta_1, \dot\theta_2$ (각속도 rad/s)
obs[8:10]	fingertip − target **오차 벡터** (m)

10차원 상태에서 읽을 세 가지

1. **각도를 (cos, sin) 쌍으로 준다** — θ 와 $\theta + 2\pi$ 는 물리적으로 같은 자세인데, 원래 각도 θ 는 $\pm\pi$ 경계에서 숫자가 급격히 점프하는 불연속 특징. cos/sin은 매끄러운 주기함수라 신경망이 "같은 자세"를 같은 입력으로 봄 — Pendulum이 $(\cos\theta, \sin\theta, \dot{\theta})$ 인 이유(Week 1, 11.2절)와 **정확히 같은 설계 원칙**: 신경망이 다루기 쉬운 특징을 주자.
2. **환경이 "오차"를 obs[8:10]으로 직접 준다** — 아래 베이스라인의 1줄 규칙 `action = -k * obs[8:10]` 이 바로 이 벡터를 씀.
3. **목표 지점은 반경 0.2 m 원판 안에서 버림표본법으로 놓임**.

왜 반경이 정확히 0.2인가 + 스텝당 보상

팔이 쭉 펴도 **0.21 m**까지밖에 닿지 않으므로 — 13.1절의 "도달 가능 영역"에서 보았듯, 목표가 팔의 도달 범위를 넘으면 과업 자체가 불가능. $0.2 < 0.21$ 의 설계는 "거의 항상 닿을 수 있다"를 보장하면서, 간간이 닿기 어려운 목표가 나와 지나치게 쉬워지지는 않게. 참고: 반경 R 원판의 균등 지점이 중심으로부터 갖는 평균 거리는 $2R/3 \approx 0.133$ m — 목표는 원점 부근에 놓인다는 뜻이지, fingertip(약 0.21 m)과의 평균 거리를 주지는 않음.

스텝당 보상은 두 항의 합 (`info` 에서 따로 볼 수 있음):

$$r = \underbrace{-1.0 \times \|p_{\text{tip}} - p_{\text{target}}\|}_{\text{거리 항}} \; \underbrace{- \; 1.0 \times \|a\|^2}_{\text{제어 비용}}$$

제어 비용 가중치 1.0: 보상 함수는 공학 제재의 번역

제어 비용의 가중치가 **1.0**이라는 점에 주의 — docstring에는 0.1이라고 적혀 있으나, 소스의 기본값은 `reward_control_weight = 1`.

문서와 코드가 다르면 코드가 맞다 — 이 분야에서 자주 일어나는 일.

"행동이 크면 벌을 주는" 항을 왜 넣나? 실물 로봇에서 큰 토크는 **모터의 부담 · 발열 · 기계 마모**이기 때문.

보상 함수는 공학 제재가 수학으로 번역되는 장소

이 항을 무시한 정책은 시뮬레이션에서 "자주 흔들어서 근접"하는 전략을 배울 수 있지만, 실물에서는 **액추에이터를 고치는** 정책.

베이스라인: 배우기 전에 네 가지 정책을 재라

PPO를 돌리기 전에, **학습을 전혀 쓰지 않는** 정책들의 리턴을 측정 — 13.1절 PD vs RL, 13.2절 PD 그리드와 같은 “베이스라인을 먼저 재라”의 습관:

```
env0 = gym.make("Reacher-v5")
print(f"무력토크( 0) 정책:  {evaluate(lambda s: np.zeros(2)):.2f}")
print(f"무작위 정책:        {evaluate(lambda s: env0.action_space.sample()):.2f}")
print(f"비례제어기 k=0.5:   {evaluate(lambda s: np.clip(-0.5 * s[8:10], -1, 1)):.2f}")
print(f"비례제어기 k=2.0:   {evaluate(lambda s: np.clip(-2.0 * s[8:10], -1, 1)):.2f}")
```

네 수를 읽는 연습: 기준선은 ≈ -9.3

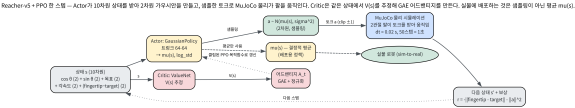
정책	10 에피소드 평균 리턴 (시드 42)
무력(토크 0)	-13.06
무작위	-44.34
비례제어기 $k = 0.5$	-9.32 $\leftarrow$ 기준선
비례제어기 $k = 2.0$	-14.57

- **무작위** ≈ -44.3: 거리 항 악화 + 제어 비용 $50 \times E[\|a\|^2] \approx 50 \times \frac{2}{3} \approx 33.3$ 이 쌓임.
- **무력** -13.1: 제어 비용 0, 초기 자세 fingertip(약 0.21 m)과 목표의 거리만 남음 (스텝당 ≈ 0.25 m $\times$ 50).
- **비례제어기 $k = 0.5$(-9.3)가 $k = 2.0$(-14.6)보다 낫다** — 이득이 크면 오히려 나빠짐. 미분 항이 없는 비례제어는 감쇠가 없어 k 가 크면 목표점을 **과충격** (overshoot)해 주위를 진동, 진동할 때마다 $\|a\|^2$ 벌 반복.

손으로 만든 1줄 규칙의 기준선 -9.3 — RL이 "사람이 쓴 규칙을 이긴다"를 보여려면 이 선을 넘어야.

실습: Reacher에서 PPO 학습시키기 — 같은 코드

Week 11.2의 Pendulum PPO 실습과 **완전히 같은 코드** (`GaussianPolicy`, `ValueNet`, GAE, 클리핑 손실)를 Reacher-v5에 그대로 적용 — **환경 이름과 입출력 차원 (10→2)만 바꾸고** 알고리즘 코드는 한 줄도 안 바꿈.



PPO 한 스텝: Actor(μ, log_std) → 샘플링 토크 clip(± 1) → MuJoCo → Critic $V(s)$ → GAE → 클립 PPO. Actor의 $\mu(s)$ 만 배포.

이 "환경 이름만 바꿔도 된다"는 경험은 **산업 표준** (역사적 일화는 21프레임에서).

먼저 짧은 실행부터 — 100 반복 × 200 스텝 (총 2만 스텝, 시드 42):

	리턴
무작위	-44.34
비례제어기 k=0.5	-9.32
PPO 2만, 결정론적(μ 만)	$-11.92 \rightarrow -12.14$
PPO 2만, 확률적(샘플링)	$-42.60 \rightarrow -52.51$

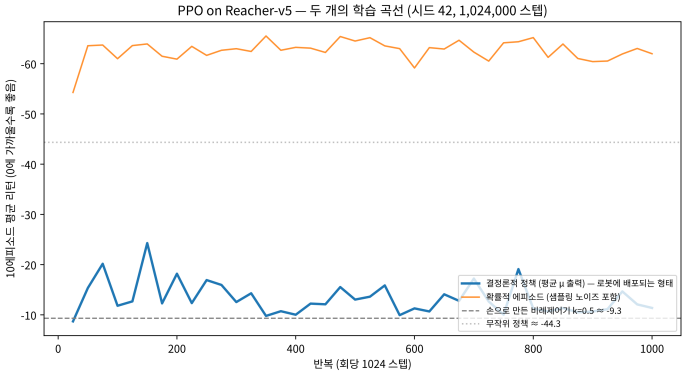
2만 스텝: 출발선일 뿐 — 그리고 100만 스텝으로

2만 스텝에서는 PPO가 아직 “출발선” — **배포 가능한** 결정론적(μ 만) 곡선은 거의 움직이지 않았고, 샘플링 노이즈가 탄 확률적 곡선은 오히려 나빠짐.

“PPO가 실패했나?”라고 생각하기 이른 것: 연속 제어 과업은 짧은 예산으로는 “아직” (11.2절의 Pendulum도 12만 스텝에 수렴 못 함). 그래서 **100만 스텝**(1000 반복 $\times$ 1024 스텝, 시드 42, 로컬 약 5분)까지 늘림:

- 결정론적(μ 만): -11.92 (학습 전) $\rightarrow -11.37$ (마지막), 중간에 $-9 \sim -17$ 사이 변동.
- 확률적(샘플링): -42.60 (학습 전) $\rightarrow -61.98$ (마지막) — **계속 악화**.
- 최종 σ: $[1.0, 1.0]$ (클램프 상한에 고정).

두 곡선 현상: 배포용 $\mu(s)$ 와 학습용 샘플



파랑 실선 = 결정론적(μ 만), 주황 = 확률적 (샘플링 포함). 회색 점선 = 비례제어기 ≈ -9.3, 은색 세로점선 = 무작위 ≈ -44.3. y축 뒤집어 위(0 근처)가 좋음.

세 가지:

- **(1) 결정론적 곡선이 손으로 만든 제어기(-9.3)와 같은 선상** — 64-64 네트워크 + 100만 스텝으로 "1줄 규칙과 어깨를 나란히"하는 팔을 학습.
- **(2) 확률적 곡선은 계속 나빠짐** — 엔트로피 보상이 "큰 σ"를 보상해 σ 를 1.0 클램프까지 밀어올림.
- **(3) 두 곡선은 같은 정책의 다른 측면**.

배포하는 것은 $\mu(s)$, 샘플이 아니다

탐색(exploration)과 활용(exploitation)은 다른 제품이다.

학습 중에는 정책이 다양한 행동 집합을 샘플링해 정보 있는 데이터를 모아야 하므로, 엔트로피 보상이 의도적으로 σ 를 부풀리는 것이 맞음. 그러나 실물 로봇에 배포되는 산출물은 **결정론적 신호** $\mu(s)$ — 로봇이 "$\sigma = 1$ 정규분포에서 토크를 뽑아 클립한다"는 식으로 움직이는 일 없음.

Week 2의 ε-greedy와 같은 구조

학습 중엔 $\varepsilon > 0$ 으로 탐색, 평가할 땐 $\varepsilon = 0$ (순수 greedy). 연속 정책에서 "greedy"가 되는 것이 곧 **평균만 출력**.

배포 단계에서 시뮬레이션(MuJoCo)을 빼버려도 됨 — 신경망 자체가 제어기. 13.1절 sim-to-real 파이프라인에서 배포 산출물이 정확히 이 "물리 시뮬레이션 없는 순수 함수 $s \mapsto \mu(s)$"라는 점.

학습된 정책을 평가하는 규칙 4가지

1. **별개의 환경 인스턴스**에서 돌리기 (학습 루프의 상태를 오염시키지 않게).
2. μ **만** 출력하기 (샘플링 안 함).
3. **고정된 시드 여러 개**(예: 10 에피소드)의 평균을 보고.
4. 확률적 곡선은 “탐색이 아직 얼마나 큰가”의 진단 지표로만 참고.

실전 워크플로

“큰 σ 로 학습 $\rightarrow$ 학습 말미에 σ 를 줄인다 (시감쇠, annealing)”거나, 배포 단계에서 샘플링을 그냥 끄고 μ 만 읽는 것.

역사적 일화: “같은 코드, 다른 환경”이 표준 레시피

“환경 이름만 바꿔도 된다”는 경험은 교육적 편의가 아니라 심층 RL의 **산업 표준**:

- PPO 논문(Schulman et al., 2017)의 실험 섹션: **같은 PPO 코드로 MuJoCo 로봇 30여 종**(Walker2D, Hopper, Ant, HalfCheetah, $\cdots$)을 훈련해 그 표를 냄 — 연속제어 시대의 벤치마크 표.
- “물리엔진 + PPO” 조합이 심층 RL의 제어 능력을 처음 검증 가능한 형태로 만듦.
- “데이터를 병렬로 모은다”의 방향은 더 이르다 — 비동기적 **A3C**가 “수많은 워커 프로세스가 트래젝터리를 병렬로 수집해 공유 정책망을 갱신한다”를 표준 레시피로 만듦.

알고리즘은 환경에서 분리된다

바로 이 구조가 2020∼2021년 GPU 대량병렬(OpenAI A1 시연, Isaac Gym “하나의 GPU로 수천 개 환경”)을 가능하게 — 알고리즘이 환경과 깔끔하게 분리되지 않았다면, 그런 시스템을 쓸 수 없게 만드는 데도 그만큼의 시간이 걸렸을. 이 책 14∼16장도 같은 구조: **알고리즘은 고정, 환경만 바꾸는 것**.

13.3 자주 하는 실수

1. **학습 중 노이즈 있는 에피소드 리턴을 정책 성능으로 판단**. 두 곡선 현상(확률적 악화, 결정론적 개선)은 오진의 전형적 원천 — “노이즈 에피소드”만 보면 “학습할수록 나빠졌다”고 결론 내고 구 weights로 돌아가는 실수. 규칙: **탐색용 학습 곡선 $\neq$ 배포 품질 곡선**.
2. **PPO가 손제어기와 비슷해지면 “구현 버그”라고 의심**. 점검 순서: (1) 예산 — 100만 스텝은 장난감, 논문은 수백만∼수천만. (2) 구조적 우위 — 1줄 제어기는 오차 벡터 obs[8:10]을 공짜로 받아 쓰는데 PPO는 시행착오로 배워야. (3) σ 클램프. 실전 표준은 손제어기를 베이스라인으로 두고 RL이 추가로 이득을 주는지 재는 것.
3. **σ 상한을 두지 않고 내버려 두기**. 클램프 없으면 엔트로피 보상이 σ 를 2, 3으로 밀어올리고, **샘플의 상당 부분이 범위 밖으로 나가 `np.clip`에게 잘림** — 이 순간 실제로 실행되는 분포는 가우시안이 아니라 잘려 변형된 분포. 클립은 “거의 발생하지 않아야” 정상 신호.

13.3 핵심 정리

핵심 정리

연속 정책은 각 행동 차원마다 가우시안의 (μ, σ) 를 출력, 로그 확률은 밀도의 로그(**양수 가능**). 그 뒤의 모든 기계 — 로그미분 트릭, 클리핑, GAE — 는 **그대로**.

배포 산출물은 결정론적 $\mu(s)$ 이고, 노이즈 있는 샘플은 학습의 수단이지 제품이 **아님** — 정책 평가는 μ 로 한다. 로봇 과업에서 PPO는 “환경 이름만 바뀐 같은 코드”이며, 손으로 만든 제어기는 경쟁자가 아니라 베이스라인.

로봇 시뮬레이션은 지금까지 배운 알고리즘(DQN, PPO)을 “실제 물리 법칙이 지배하는 연속 행동공간”이라는 더 현실적인 무대로 옮기는 시험대 — 다음 장에서는 이 시뮬레이션을 더 정교한 물리엔진으로, 그리고 대규모 GPU 가속 환경으로 확장.

이번 주차 정리

1. **13.1 로봇공학과 Sim-to-Real** — 상태(각도·각속도)· 행동(토크)은 물리적 단위의 연속값. 시뮬레이션 = 동역학 미분방정식을 dt 로 끊어 반복. **현실 격차**는 편이(bias)라 데이터로 안 줄고, **도메인 무작위화** + **시스템 ID**로 줄임. "시뮬레이션 최고점" $\neq$ "실물 최적".
2. **13.2 로봇 환경 탐색** — 실습의 첫 단계는 알고리즘이 아니라 **평가 프로토콜**(창 고정, 다중 시드, 리턴+길이+종료 비율). PD 그리드는 2차원에서 가능하나 105$\rightarrow$8 고차원에서는 형식적으로 죽음. 보행 보상은 **4항 장부**.
3. **13.3 PPO 연속제어** — 가우시안 정책 (μ, σ) 로 PPO를 확장, Reacher에서 100만 스텝으로 1줄 제어기 수준. **배포 산출물 =** $\mu(s)$. 손제어기는 베이스라인.

다음 주차 — Week 14. 고급 시뮬레이션: MuJoCo와 Isaac Sim

이번 장의 Gymnasium 기본 환경은 물리를 간단한 근사식으로 계산해 빠르지만 접촉·마찰은 거칠게만 흉내 냄. 다음 장에서는 이 물리 엔진을 **실전 로봇 연구 수준**으로 한 단계 올려본다.